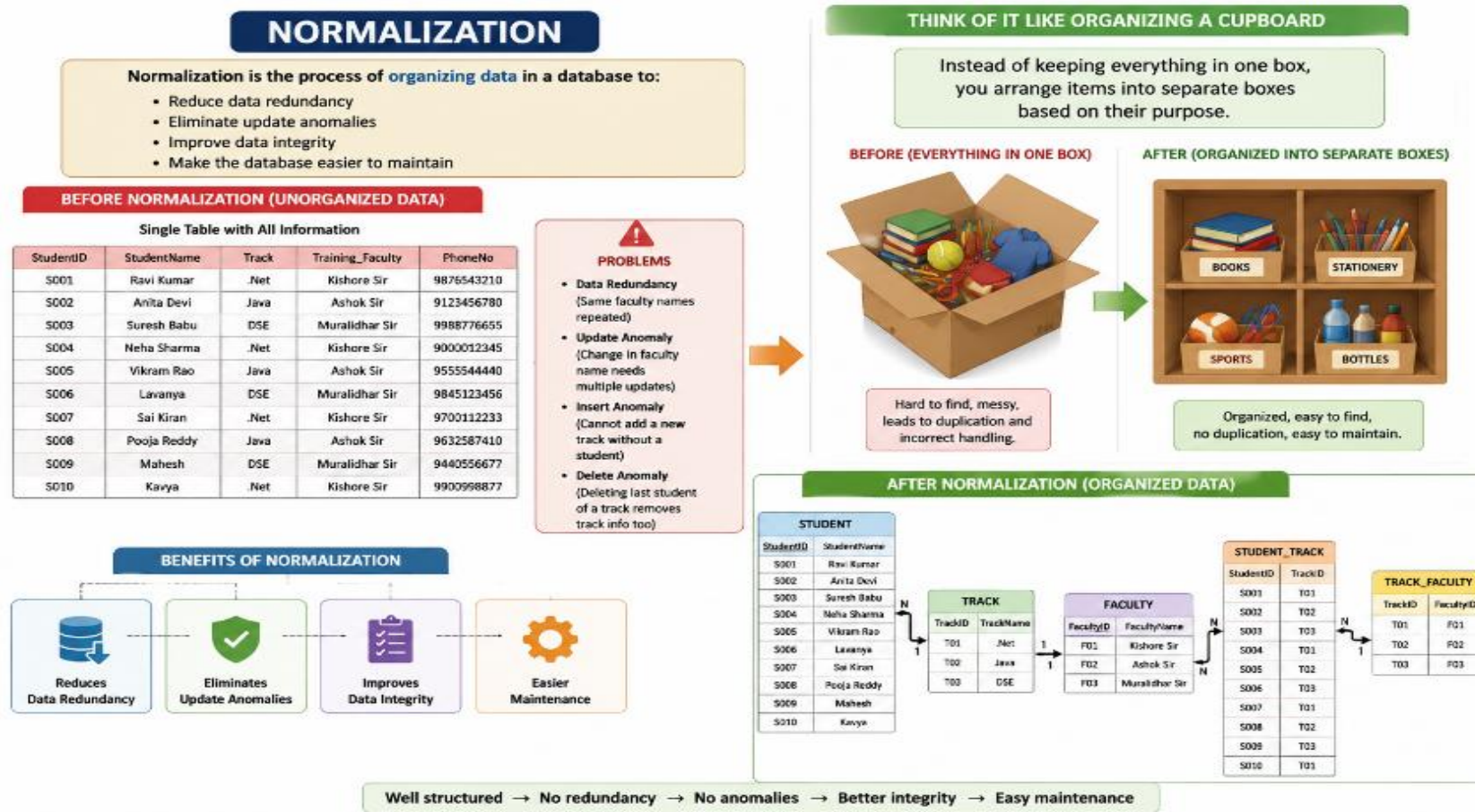


NORMALIZATION

Normalization is the process of **organizing data in a database to:**

- Reduce data redundancy
- Eliminate update anomalies
- Improve data integrity
- Make the database easier to maintain



Think of it like **organizing a cupboard**. Instead of keeping everything in **one box**, you arrange items into separate boxes based on their purpose.

Example:
WTN2026
Student

Track-Faculty Mapping

WTN2026 Track	Training Faculty	Phone Number
.NET	Kishore Sir	9876543210
Java	Ashok Sir	9123456789
DSE	Muralidhar Sir	9988776655

STEP 0: UNNORMALIZED FORM (UNF)



Notice that some students have enrolled in multiple tracks, so multiple values are stored in the same cell.

Student ID	Student Name	WTN2026 Tracks	Training Faculty	Faculty Phone
101	Rahul	.NET, Java	Kishore Sir, Ashok Sir	9876543210, 9123456789
102	Sneha	Java	Ashok Sir	9123456789
103	Kiran	.NET, DSE	Kishore Sir, Muralidhar Sir	9876543210, 9988776655
104	Anjali	Java, DSE	Ashok Sir, Muralidhar Sir	9123456789, 9988776655
105	Vamsi	.NET	Kishore Sir	9876543210
106	Harsha	Java	Ashok Sir	9123456789
107	Divya	DSE	Muralidhar Sir	9988776655
108	Nikhil	.NET, Java	Kishore Sir, Ashok Sir	9876543210, 9123456789
109	Meghana	DSE, Java	Muralidhar Sir, Ashok Sir	9988776655, 9123456789
110	Sai	.NET	Kishore Sir	9876543210



PROBLEMS

- ✗ Multiple tracks in one column
- ✗ Multiple faculty names in one column
- ✗ Multiple phone numbers in one column
- ✗ Difficult to search and update



This is
UNF.

STEP 1: FIRST NORMAL FORM (1NF)



RULE

One value per cell.
Split every
multi-valued row.



Student ID	Student Name	Track	Faculty	Phone
101	Rahul	.NET	Kishore Sir	9876543210
101	Rahul	Java	Ashok Sir	9123456789
102	Sneha	Java	Ashok Sir	9123456789
103	Kiran	.NET	Kishore Sir	9876543210
103	Kiran	DSE	Muralidhar Sir	9988776655
104	Anjali	Java	Ashok Sir	9123456789
104	Anjali	DSE	Muralidhar Sir	9988776655
105	Vamsi	.NET	Kishore Sir	9876543210
106	Harsha	Java	Ashok Sir	9123456789
107	Divya	DSE	Muralidhar Sir	9988776655
108	Nikhil	.NET	Kishore Sir	9876543210
108	Nikhil	Java	Ashok Sir	9123456789
109	Meghana	DSE	Muralidhar Sir	9988776655
109	Meghana	Java	Ashok Sir	9123456789
110	Sai	.NET	Kishore Sir	9876543210



There are now
15 rows, but
each cell
contains only
one value.



1NF
achieved.

So 1 NF Achieved .. But still Problem exists, what are they ... Repeated data... is there ?

Examine 1 NF Table

PROBLEMS AFTER 1NF



Observe the repeated data.

TABLE AFTER 1NF

Student ID	Student Name	Track	Faculty	Phone
101	Rahul	.NET	Kishore Sir	9876543210
101	Rahul	Java	Ashok Sir	9123456789
102	Sneha	Java	Ashok Sir	9123456789
103	Kiran	.NET	Kishore Sir	9876543210
103	Kiran	DSE	Muralidhar Sir	9988776655
104	Anjali	Java	Ashok Sir	9123456789
104	Anjali	DSE	Muralidhar Sir	9988776655
105	Vamsi	.NET	Kishore Sir	9876543210
106	Harsha	Java	Ashok Sir	9123456789
107	Divya	DSE	Muralidhar Sir	9988776655
108	Nikhil	.NET	Kishore Sir	9876543210
108	Nikhil	Java	Ashok Sir	9123456789
109	Meghana	DSE	Muralidhar Sir	9988776655
109	Meghana	Java	Ashok Sir	9123456789
110	Sai	.NET	Kishore Sir	9876543210

1 Student ID and Student Name are repeated

Student ID	Student Name
101	Rahul
101	Rahul



Student name repeats.

2 Faculty and Phone are repeated

Faculty	Phone
Kishore Sir	9876543210
Kishore Sir	9876543210
Kishore Sir	9876543210



Phone numbers are repeated many times.



Problems after 1NF:

- ✓ Data redundancy (repeated data)
- ✓ Update anomaly (if phone number changes, we must update in many rows)
- ✓ Insertion anomaly (cannot easily insert a faculty without student details)
- ✓ Deletion anomaly (deleting a student may remove faculty information)



We need to remove these redundancies.
Move to 2NF.

STEP 2: SECOND NORMAL FORM (2NF)



Assume the **Primary Key** = (StudentID, Track)

- ➔ **StudentName** depends only on **StudentID**.
- ➔ It does not depend on **Track**.
- ➔ Therefore remove **partial dependency**.

Why is this a problem in 1NF?

- StudentName repeats for every track.
- If a student's name changes, we must update multiple rows.



GOAL

Remove partial dependency by separating Student details from Enrollment (Track) details.

This gives us **2NF**.

BEFORE (1NF TABLE)

Student ID	Student Name	Track	Faculty	Phone
101	Rahul	.NET	Kishore Sir	9876543210
101	Rahul	Java	Ashok Sir	9123456789
102	Sneha	Java	Ashok Sir	9123456789
103	Kiran	.NET	Kishore Sir	9876543210
103	Kiran	DSE	Muralidhar Sir	9988776655
104	Anjali	Java	Ashok Sir	9123456789
104	Anjali	DSE	Muralidhar Sir	9988776655
105	Vamsi	.NET	Kishore Sir	9876543210
106	Harsha	Java	Ashok Sir	9123456789
107	Divya	DSE	Muralidhar Sir	9988776655
108	Nikhil	.NET	Kishore Sir	9876543210
108	Nikhil	Java	Ashok Sir	9123456789
109	Meghana	DSE	Muralidhar Sir	9988776655
109	Meghana	Java	Ashok Sir	9123456789
110	Sai	.NET	Kishore Sir	9876543210

Remove partial dependency



What changed?

StudentName moved to its own table because it depends only on StudentID, not on Track.

AFTER (2NF) – TWO TABLES

1) STUDENT TABLE

Student ID	Student Name
101	Rahul
102	Sneha
103	Kiran
104	Anjali
105	Vamsi
106	Harsha
107	Divya
108	Nikhil
109	Meghana
110	Sai

2) ENROLLMENT TABLE

Student ID	Track	Faculty	Phone
101	.NET	Kishore Sir	9876543210
101	Java	Ashok Sir	9123456789
102	Java	Ashok Sir	9123456789
103	.NET	Kishore Sir	9876543210
103	DSE	Muralidhar Sir	9988776655
104	Java	Ashok Sir	9123456789
104	DSE	Muralidhar Sir	9988776655
105	.NET	Kishore Sir	9876543210
106	Java	Ashok Sir	9123456789
107	DSE	Muralidhar Sir	9988776655
108	.NET	Kishore Sir	9876543210
108	Java	Ashok Sir	9123456789
109	DSE	Muralidhar Sir	9988776655
109	Java	Ashok Sir	9123456789
110	.NET	Kishore Sir	9876543210



BENEFITS OF 2NF



Student details are stored only once.



Update anomaly is removed for StudentName.



Insertion anomaly is improved.



Deletion anomaly is improved.



STUDENT details are stored only once.

Partial dependency is removed.



2NF achieved!

Outcome of 1 NF and 2NF Together

PROBLEMS AFTER 1NF



Observe the repeated data in 1NF table.

TABLE AFTER 1NF (15 ROWS)

Student ID	Student Name	Track	Faculty	Phone
101	Rahul	.NET	Kishore Sir	9876543210
101	Rahul	Java	Ashok Sir	9123456789
102	Sneha	Java	Ashok Sir	9123456789
103	Kiran	.NET	Kishore Sir	9876543210
103	Kiran	DSE	Muralidhar Sir	9988776655
104	Anjali	Java	Ashok Sir	9123456789
104	Anjali	DSE	Muralidhar Sir	9988776655
105	Vamsi	.NET	Kishore Sir	9876543210
106	Harsha	Java	Ashok Sir	9123456789
107	Divya	DSE	Muralidhar Sir	9988776655
108	Nikhil	.NET	Kishore Sir	9876543210
108	Nikhil	Java	Ashok Sir	9123456789
109	Meghana	DSE	Muralidhar Sir	9988776655
109	Meghana	Java	Ashok Sir	9123456789
110	Sai	.NET	Kishore Sir	9876543210

WHAT ARE THE PROBLEMS?

1 Student ID and Student Name are repeated

Student ID	Student Name
101	Rahul
101	Rahul

Student name repeats.

2 Faculty and Phone are repeated

Faculty	Phone
Kishore Sir	9876543210
Kishore Sir	9876543210
Kishore Sir	9876543210

Phone numbers are repeated many times.

Impacts of Repetition (Redundancy)

- Data redundancy (repeated data)
- Update anomaly (if phone number changes, we must update in many rows)
- Insertion anomaly (cannot easily insert a faculty without student details)
- Deletion anomaly (deleting a student may remove faculty information)

IN CONNECTION, WHAT SHOULD WE DO?

To solve these problems, we remove redundancy by separating data into appropriate tables. This is done in the next normal form (2NF).

APPROACH

- 1 Identify repeating groups.
- 2 Create separate table for Faculty details (because Faculty → Phone is independent of Student).
- 3 Keep Student – Track – Faculty relationship in another table.
- 4 Remove redundancy and maintain relationships using keys.

RESULT Data is organized, no repetition, and integrity is improved. Move to 2NF.

NEXT STEP: STEP 2 – SECOND NORMAL FORM (2NF)



RULE OF 2NF

Table must be in 1NF and Every non-key attribute must be fully dependent on the whole primary key, not on a part of it.

HOW WE CONVERT 1NF TO 2NF?

Separate the data into two tables:

- 1 STUDENT_TRACK
(contains student, track and faculty)



- 2 FACULTY
(contains faculty and phone)

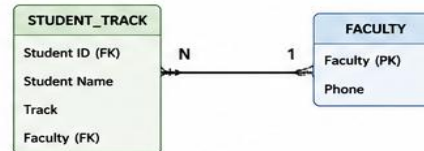
1) STUDENT_TRACK TABLE (15 ROWS)

Student ID	Student Name	Track	Faculty
101	Rahul	.NET	Kishore Sir
101	Rahul	Java	Ashok Sir
102	Sneha	Java	Ashok Sir
103	Kiran	.NET	Kishore Sir
103	Kiran	DSE	Muralidhar Sir
104	Anjali	Java	Ashok Sir
104	Anjali	DSE	Muralidhar Sir
105	Vamsi	.NET	Kishore Sir
106	Harsha	Java	Ashok Sir
107	Divya	DSE	Muralidhar Sir
108	Nikhil	.NET	Kishore Sir
108	Nikhil	Java	Ashok Sir
109	Meghana	DSE	Muralidhar Sir
109	Meghana	Java	Ashok Sir
110	Sai	.NET	Kishore Sir

2) FACULTY TABLE (DISTINCT) (3 ROWS)

Faculty	Phone
Kishore Sir	9876543210
Ashok Sir	9123456789
Muralidhar Sir	9988776655

RELATIONSHIP BETWEEN TABLES



Meaning:

- One faculty can train many students (1 to N).
- Each student is trained by one faculty.



BENEFITS AFTER 2NF



No repetition of faculty phone numbers.



Update anomaly removed (Change phone in one place only).



Insertion anomaly removed (Can add a faculty without student details).



Deletion anomaly removed (Deleting a student will not delete faculty info).



Data integrity improved.



With 2NF, we have removed redundancy related to partial dependencies. Next, we move to 3NF to remove transitive dependencies. 

Converting 2 NF to 3 NF:

STEP 3: PROBLEMS AFTER 2NF



Notice this dependency:

Track → Faculty

Faculty → Phone

Therefore,

Track → Faculty → Phone



Phone depends on Faculty,
not directly on Track.



This is a
Transitive Dependency.

Example from 2NF (ENROLLMENT Table)

Student ID	Track	Faculty	Phone
101	.NET	Kishore Sir	9876543210
101	Java	Ashok Sir	9123456789
102	Java	Ashok Sir	9123456789
103	.NET	Kishore Sir	9876543210
103	DSE	Muralidhar Sir	9988776655
104	Java	Ashok Sir	9123456789
104	DSE	Muralidhar Sir	9988776655
105	.NET	Kishore Sir	9876543210
106	Java	Ashok Sir	9123456789
107	DSE	Muralidhar Sir	9988776655
108	.NET	Kishore Sir	9876543210
108	Java	Ashok Sir	9123456789
109	DSE	Muralidhar Sir	9988776655
109	Java	Ashok Sir	9123456789
110	.NET	Kishore Sir	9876543210

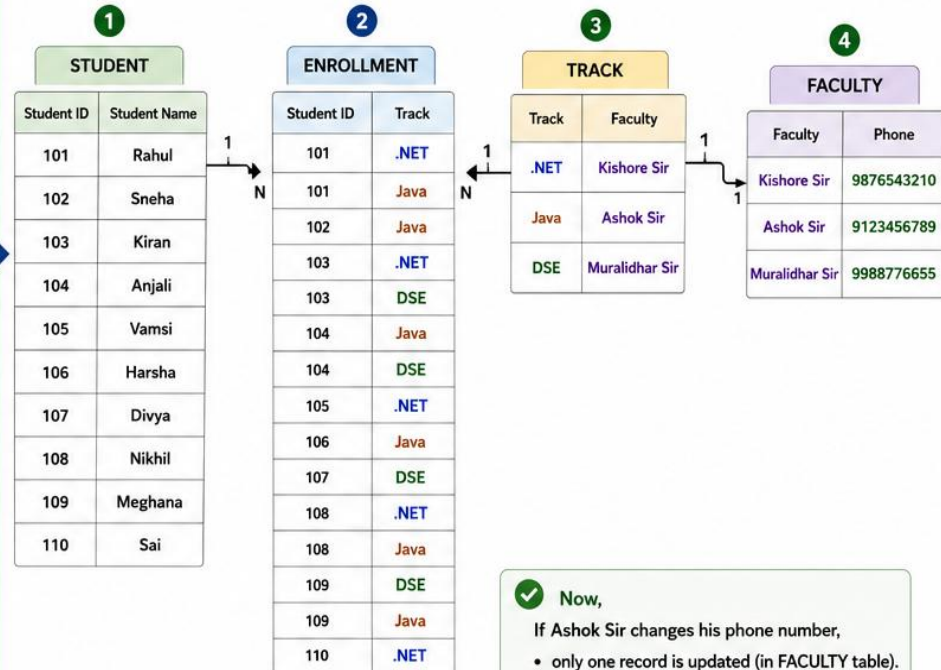


Phone numbers are repeated because Phone
depends on Faculty (transitively on Track).

STEP 3: THIRD NORMAL FORM (3NF)



Split Faculty information to remove transitive dependency.



Now,

- If Ashok Sir changes his phone number,
- only one record is updated (in FACULTY table).
- No repeated phone numbers.

3NF achieved.

BENEFITS OF 3NF



No redundancy
(No repeated
phone numbers)



Update anomaly
removed
(Update in one place
only)



Insertion anomaly
removed
(Can add faculty
without students)



Deletion anomaly
removed
(Deleting a student
won't remove faculty
or phone info)



Data integrity
improved
(Clear relationships
among entities)



We removed transitive dependency by splitting Faculty information. ➡ Database is now in 3NF – clean, efficient, and easy to maintain.

Converting to 3 NF to BCNF :

STEP 4: BCNF (BOYCE-CODD NORMAL FORM)



The dependency is:

Track → **Faculty**

Since each Track has exactly one Faculty, store it only in the **TRACK** table.



WHY BCNF?

- In BCNF, for every dependency $X \rightarrow Y$, X must be a super key.
- Here, in **TRACK** table, Track is the primary key and determines Faculty.
- No other dependencies exist in each table.



RESULT

All tables are in BCNF.
BCNF achieved.

HOW WE REACH BCNF

From 3NF Design

STUDENT		ENROLLMENT		TRACK		FACULTY	
Student ID	Student Name	Student ID	Track	Track	Faculty	Faculty	Phone
101	Rahul	101	.NET	.NET	Kishore Sir	Kishore Sir	9876543210
102	Sneha	101	Java	Java	Ashok Sir	Ashok Sir	9123456789
103	Kiran	102	Java				
104	Anjali	103	.NET	DSE	Muralidhar Sir	Muralidhar Sir	9988776655
105	Vamsi	103	DSE				
106	Harsha	104	Java				
107	Divya	104	DSE				
108	Nikhil	105	.NET				
109	Meghana	106	Java				
110	Sai	107	DSE				
		108	.NET				
		108	Java				
		109	DSE				
		109	Java				
		110	.NET				



Dependency in this design: **Track** → **Faculty**

Track is the primary key of **TRACK** table. So, dependency is valid and in BCNF.

CHECKING BCNF RULE

For every non-trivial dependency $X \rightarrow Y$ in each table, X must be a super key.



STUDENT table

No non-trivial dependencies.
So, BCNF.



ENROLLMENT table

No non-trivial dependencies.
So, BCNF.



TRACK table

Track → **Faculty**

Track is the Primary Key (super key).
So, BCNF.



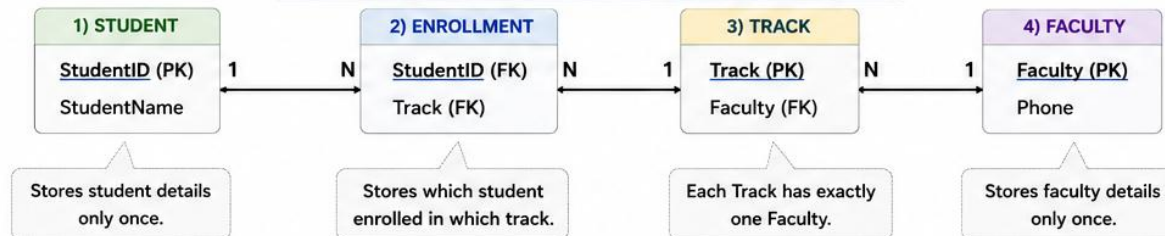
FACULTY table

No non-trivial dependencies.
So, BCNF.



All tables satisfy BCNF.
BCNF achieved.

FINAL DATABASE DESIGN (BCNF)



BENEFITS OF BCNF



No data redundancy



Update anomaly eliminated



Insertion anomaly eliminated



Deletion anomaly eliminated



Data integrity maximized

UseCase 1:



HOMEWORK ASSIGNMENT – 1

Use Case: College Library Management System



Goal

Normalize the given data from UNF to BCNF and design a proper database schema.

SCENARIO

VVITU Central Library maintains records of students, books, authors, librarians and book borrowing details. Currently all information is stored in a single table, which contains repeated data and causes anomalies.

INITIAL DATA (UNF)

Student ID	Student Name	Borrowed Books	Author Names	Librarian	Librarian Phone
101	Rahul	DBMS, Operating Systems	Korth, Galvin	Lakshmi Madam	9876543210
102	Sneha	Java Programming	Herbert Schildt	Ramesh Sir	9123456789
103	Kiran	Python, DBMS	Mark Lutz, Korth	Lakshmi Madam	9876543210
104	Anjali	Computer Networks, DBMS	Forouzan, Korth	Suresh Sir	9988776655
105	Vamsi	Java Programming, Python	Herbert Schildt, Mark Lutz	Ramesh Sir	9123456789

Why UNF? (Possible Problems)

- Multiple books in one cell
- Multiple authors in one cell
- Librarian phone repeated many times
- Data redundancy
- Update, Insertion and Deletion anomalies
- Difficult to maintain and search

REAL WORLD OBJECTS



STUDENT



BOOK



LIBRARIAN



BORROWING TRANSACTION

EXPECTED FINAL TABLES

STUDENT

BOOK

BORROW

LIBRARIAN



Normalized, Non-Redundant,
Consistent and Easy to Maintain

STUDENT TASKS (Normalization Steps)

1 Analyze the UNF Table

- Identify all problems in the given table.
- Explain why it is in Unnormalized Form (UNF).
- Mention the anomalies that may occur.



2 Convert UNF to First Normal Form (1NF)

- Apply the rules of 1NF.
- Remove all repeating groups.
- Ensure each cell has only one value.
- Draw the resulting 1NF table.



3 Identify the Primary Key

- Determine the appropriate primary key for the 1NF table.
- Justify your choice.



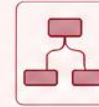
4 Convert 1NF to Second Normal Form (2NF)

- Identify all partial dependencies.
- Remove partial dependencies.
- Create appropriate relations (tables).
- Mention the primary key of each relation.



5 Convert 2NF to Third Normal Form (3NF)

- Identify all transitive dependencies.
- Remove transitive dependencies.
- Create the required relations (tables).
- Specify primary key and foreign key for each table.



6 Convert 3NF to Boyce-Codd Normal Form (BCNF)

- Verify whether every determinant is a super key.
- If necessary, decompose the relations.
- Explain why the final design satisfies BCNF.



7 Functional Dependencies

- For every normalization step, list the functional dependencies used.



8 Final Database Schema

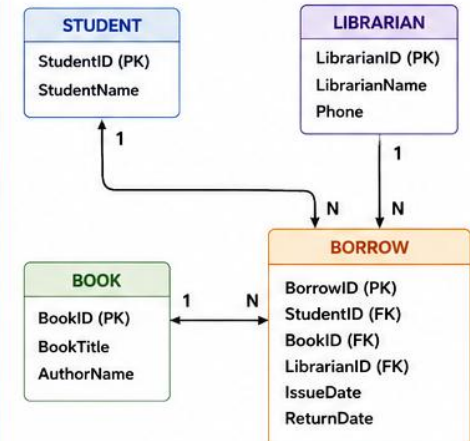
- Draw the final normalized database schema.
- Show tables with Primary Key (PK) and Foreign Key (FK).
- Display relationships among the tables.



MARKS DISTRIBUTION (20 MARKS)

Question	Marks
1. UNF Problems	2
2. 1NF	3
3. Primary Key Identification	2
4. 2NF	4
5. 3NF	4
6. BCNF	3
7. Functional Dependencies	2
8. Final Schema Diagram	2
TOTAL	20

EXPECTED FINAL TABLES (AFTER BCNF)



NOTE

Work step-by-step. Show all intermediate tables, keys, dependencies and justifications clearly.

UseCase 2:

SCENARIO

A hospital stores patient appointments in one table. All information about patients, departments, doctors and phone numbers is mixed together.

INITIAL DATA (UNF)

PatientID	PatientName	Departments	Doctors	Doctor Phone
P101	Ravi	Cardiology, Neurology	Dr. Kumar, Dr. Suresh	9876543210, 9123456789
P102	Priya	Orthopedics	Dr. Ashok	9988776655
P103	Maresh	Cardiology, Pediatrics	Dr. Kumar, Dr. Anitha	9876543210, 9012345678
P104	Divya	Neurology	Dr. Suresh	9123456789
P105	Harsha	Orthopedics, Cardiology	Dr. Ashok, Dr. Kumar	9988776655, 9876543210

DOCTOR-DEPARTMENT MAPPING

Department	Doctor
Cardiology	Dr. Kumar
Neurology	Dr. Suresh
Orthopedics	Dr. Ashok
Pediatrics	Dr. Anitha

EXPECTED OUTCOME



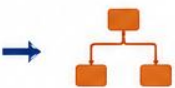
Students will normalize the given UNF table step-by-step up to BCNF, identify keys, draw ER diagram, list functional dependencies and explain database anomalies.



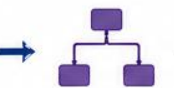
UNF
Redundant Data & Anomalies



1NF
Atomic Values



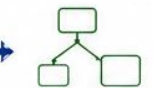
2NF
No Partial Dependencies



3NF
No Transitive Dependencies



BCNF
Every Determinant is a Super Key



ER DIAGRAM
Clear Structure & Relationships



HOMWORK ASSIGNMENT – 2
Use Case: Hospital Appointment Management System

STUDENT TASKS

1

Convert the above table into 1NF.

- Ensure every cell contains only one value.



2

Identify the Composite Primary Key.

- Decide the primary key for the 1NF table.



3

Remove Partial Dependencies (2NF).

- Identify all partial dependencies.
- Decompose the table to remove them.



4

Remove Transitive Dependencies (3NF).

- Identify all transitive dependencies.
- Decompose the tables to remove them.



5

Check whether the database satisfies BCNF.

- For every non-trivial dependency $X \rightarrow Y$, X must be a super key.



6

Draw the ER Diagram.

- Show entities, attributes, primary keys and relationships.



7

List all Functional Dependencies.

- List the functional dependencies used in each step.



8

Explain Anomalies (Using given data).

- Explain one Insertion Anomaly.
- Explain one Update Anomaly.
- Explain one Deletion Anomaly.



EXPECTED FINAL TABLES
(AFTER BCNF)

PATIENT

PatientID (PK)
PatientName

DEPARTMENT

DepartmentID (PK)
DepartmentName

DOCTOR

DoctorID (PK)
DoctorName
DoctorPhone

APPOINTMENT

PatientID (FK)
DepartmentID (FK)
DoctorID (FK)
AppointmentDate
AppointmentTime

MARKS DISTRIBUTION (20 MARKS)

Question	Marks
1. 1NF	3
2. Composite Primary Key	2
3. 2NF (Remove Partial Dependencies)	4
4. 3NF (Remove Transitive Dependencies)	4
5. BCNF Check	2
6. ER Diagram	2
7. Functional Dependencies	2
8. Anomalies (Insert, Update, Delete)	1
TOTAL	20



NOTE

Show all intermediate tables for each normalization step and justify your answers clearly.

